

**Building and Verifying a Replicated Counter Service: Communication, Logical Time,
and Quorum Agreement**

Danil Shumayev

School of Digital Technologies, Cybersecurity, 4th Year

Distributed Systems

Ernazaruly Bakdaulet

October 9, 2026

Building and Verifying a Replicated Counter Service: Communication, Logical Time, and Quorum Agreement

Design Summary

RPC Semantics (Task A2)

The design gives at-most-once execution of each write. The client creates one idempotency key (uuid4) per user action and reuses it on every retry and on every replica. Each replica stores the result of every key it has applied and answers a repeated key from this store with `was_duplicate = true`, without adding the delta again. After a timeout the client cannot know whether the request or the reply was lost, but a retry with the same key changes the counter exactly once in both cases (TC-05, TC-14).

Concurrency (Task A3)

Each server runs a thread pool with eight workers. One threading.Lock protects all shared state (counter values and the deduplication store); the duplicate check and the update run inside the same locked block (TC-03, TC-10).

Quorum Writes and Reads (Task C1)

A write is sent to all three replicas and is committed only if at least two acknowledge it. A failed write can still be partly applied: in TC-07 the client reports failure, but the one live replica has already applied +5 and would show it to readers. Raft prevents this because a replica applies an entry only after the leader knows a majority has stored it; entries that never reach a majority are overwritten.

Get reads one replica, so it can return a stale value, for example from a replica that was just restarted (TC-17: B shows 0 while the committed value is 30).

Requirements Under Test

Table 1

Requirements Under Test

Task	Requirement	Verified by
A2	Deadline on every call, up to 3 retries with backoff, same idempotency key on retries, server suppresses duplicates	TC-02, TC-05, TC-10, TC-13, TC-14
A3	Exact result under concurrent clients; dedup check and update under one lock	TC-03, TC-10
B1	Lamport rules: +1 before each local event, $\max(\text{own}, \text{received}) + 1$ on receive; one log line per event	TC-11; captured trace in Appendix C
C1	Write commits only with at least 2 of 3 acknowledgements; failure is reported below majority	TC-06, TC-07, TC-08, TC-12, TC-15, TC-16, TC-17
A1/A2	Basic Increment and Get behaviour	TC-01, TC-04, TC-09

Test Environment

Table 2*Test Environment*

Item	Value
Machine	Apple MacBook Air (Mac16,12), Apple M4, 10 CPU cores, 16 GB RAM
Operating system	macOS 27.2 (build 26B5091g), arm64
Python	3.14.7 (Homebrew), virtual environment .venv
Key packages	grpcio 1.84.0, grpcio-tools 1.84.0, protobuf 7.36.2, pytest 9.1.1 (full list in Appendix D)
Network	All processes on one machine, connected through localhost

Test Cases**Table 3***Test Cases (17 of 17 Passed)*

ID	Test	Purpose	Expected	Actual	Result
TC-01	test_increment_applies_delta	One Increment adds its delta	Committed, value 5	new_value = 5, Get = 5	Pass
TC-02	test_duplicate_key_not_reapplied	Same key sent twice	Second reply 5, was_duplicate = true	5 then 5 (duplicate); not 10	Pass
TC-03	test_concurrent_increment_exact	2 threads $\times$ 1,000 increments	Exactly 2,000	2,000	Pass
TC-04	test_get_missing_counter	Get on unknown counter	found = false, no exception	found = false	Pass
TC-05	test_retry_after_timeout_is_safe	Reply delayed 800 ms, deadline 300 ms	Retry succeeds, counter moves once	value 1, retry answered as duplicate, 1 APPLY in log	Pass
TC-06	test_majority_commit_two_acks	Replica B stopped	Committed with 2 acks	committed, 2 acks, A = C = 1	Pass
TC-07	test_no_commit_below_majority	Replicas B and C stopped	Not committed, no value reported	committed = false, 1 ack, new_value = None	Pass
TC-08	test_replicas_converge	30 quorum writes on 3 counters	All replicas identical	A = B = C = {c0: 10, c1: 10, c2: 10}	Pass
TC-09	test_counter_ids_are_isolated	Two counters	a = 3, b = 10	a = 3, b = 10	Pass
TC-10	test_retry_storm_same_key_applied_once	20 concurrent retries of one operation	Applied once on each replica	7 on A, B, C; one APPLY each	Pass
TC-11	test_lamport_clock_rules	tick, then receive 10, then receive 3	1, 11, 12	1, 11, 12	Pass

ID	Test	Purpose	Expected	Actual	Result
TC-12	test_replica_crash_mid_request	B killed while the write is in flight	Commits with 2 acks, no exception	committed, 2 acks, A = C = 1	Pass
TC-13	test_request_duplication	Same key twice, 3 replica processes	Duplicate flagged, value moves once	was_duplicate = true, A = B = C = 5	Pass
TC-14	test_induced_timeout_with_retry	B replies after 1,500 ms, deadline 500 ms	Retry succeeds, B applies once	3 acks, B = 1; B log: 2 RECV, 1 APPLY, 1 DUP	Pass
TC-15	test_crash_before_and_after_apply [before]	B exits before applying	Commits 2/3, B did not apply	committed, 2 acks, no APPLY in B log	Pass
TC-16	test_crash_before_and_after_apply [after]	B applies, then exits before replying	Commits 2/3, B applied but did not ack	committed, 2 acks, APPLY in B log	Pass
TC-17	test_restart_does_not Lose committed writes	3 rounds: kill B, write, restart B	All 30 writes committed, none lost on A and C	30 committed, A = C = 30, B = 0	Pass

Note. TC-01 to TC-11 are in tests/test_counter.py (replicas in-process); TC-12 to TC-17 are in tests/test_failures.py (replicas are real processes started with subprocess.Popen and stopped with proc.kill()). Full suite: 17 passed in 5.7 s; six consecutive runs had no failures.

Failure-Injection Results

Replicas are real processes started with subprocess.Popen and stopped with proc.kill(). Server flags --delay-ms and --fault crash-before-apply / crash-after-apply inject the faults.

Table 4

Failure-Injection Results

Scenario	Injected fault	Invariant asserted	Evidence (replica-B log unless noted)
Crash mid-request (TC-12)	B started with --delay-ms 1000, killed with proc.kill() after 0.3 s	Write commits with 2 acks; no exception reaches the caller	APPLY counter=x -> 1 L=4 FAULT reply delayed 1000 ms L=5 (no SEND: process killed)
Request duplication (TC-13)	Same key sent twice	Second reply has was_duplicate = true; value moves once	DUP key=dup-key already applied -> 5 L=14
Timeout with retry (TC-14)	B started with --delay-ms 1500; client deadline 0.5 s	Retry succeeds; B applies the write once	APPLY counter=x -> 1 L=4 RECV Increment(...) L=10 (received L=9) DUP key=92a63a98 already applied -> 1 L=11
Crash before apply (TC-15)	--fault crash-before-apply	Commits 2/3; B never applied	RECV Increment(...) L=3 FAULT crash-before-apply: process exits L=4

Scenario	Injected fault	Invariant asserted	Evidence (replica-B log unless noted)
Crash after apply (TC-16)	--fault crash-after-apply	Commits 2/3; B applied but never acked	APPLY counter=x -> 1 L=5 FAULT crash-after-apply: process exits L=6
Below majority (TC-07)	B and C stopped	Not committed; value is not reported	replica-A: APPLY counter=x -> 6 L=12 (partial apply)
Restarts (TC-17)	B killed and restarted three times	Every committed write survives on A and C	A = C = 30 after 30 commits; B = 0 after its last restart

Performance

Script: tests/perf_benchmark.py. Latency of one client Increment call; 2,000 requests per configuration, three runs each.

Table 5

Increment Latency (Median of Three Runs)

Configuration	Median latency (ms)	p95 latency (ms)	Requests
Single replica, 1 client	0.16	0.18	2,000
Single replica, 16 clients	1.72	2.60	2,000
Quorum (3 replicas), 1 client	0.29	0.33	2,000
Quorum (3 replicas), 16 clients	3.72	6.71	2,000

Note. p95 is the value at index $\text{ceil}(0.95 \times n)$ of the sorted latencies. Each run starts fresh replica processes and sends 50 warm-up requests per client that are not counted.

Table 6

Repeated Runs and Throughput

Configuration	Median per run (ms)	p95 per run (ms)	Throughput (req/s), mean $\pm$ SD
Single replica, 1 client	0.16, 0.16, 0.16	0.19, 0.17, 0.18	6,203 $\pm$ 89
Single replica, 16 clients	1.72, 1.73, 1.69	2.66, 2.60, 2.57	8,847 $\pm$ 122
Quorum (3 replicas), 1 client	0.30, 0.29, 0.29	0.33, 0.33, 0.33	3,372 $\pm$ 15
Quorum (3 replicas), 16 clients	3.68, 3.72, 3.73	6.67, 6.71, 6.91	3,964 $\pm$ 24

The quorum path is about 1.8 times slower with one client and 2.2 times slower with 16 clients. The extra time comes from sending each write to three replicas and waiting for all three replies, so a write costs the slowest of three round trips. With 16 clients the three server processes and the client threads also compete for the CPU, which raises p95 and halves throughput (8,847 to 3,964 requests per second). For a like counter a few milliseconds are acceptable, because in return no committed like is lost when one replica crashes (TC-06, TC-17).

Known Limitations

1. State is in memory only: a restarted replica comes back empty and does not catch up (TC-17). A write-ahead log and log replication, as in Raft (Weeks 6–7), would fix this.
2. A write that fails below majority can stay applied on a minority (TC-07); there is no commit index or rollback.
3. Get reads one replica, so it can return stale or uncommitted values.
4. The benchmark runs on one machine over localhost with client threads in one Python process, so real network delay is missing and the GIL adds latency. Because of the GIL, the concurrency test also cannot reliably detect a missing lock.

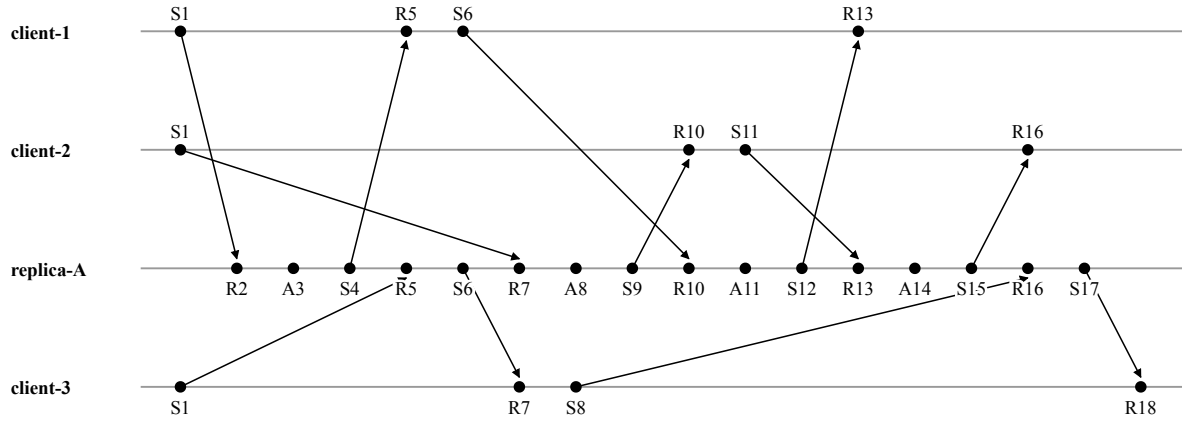
Appendix A

Ordering Analysis (Task B2)

client-1 increments x twice, client-2 increments y twice at the same time, client-3 calls Get(x) twice. The merged trace is in Appendix C and logs/b2_merged.log.

Figure 1

Happened-Before Diagram of the Captured Trace



Note. S = send, R = receive, A = apply; the number is the Lamport value L. Arrows are messages; events on one line are in local order.

Causal pair 1. client-1 SEND (L=1) → client-3 RECV GetReply(value=1) (L=7).

Chain: client-1 SEND (1) → replica-A RECV (2) → APPLY (3) → SEND (4) → RECV Get (5) → SEND GetReply (6) → client-3 RECV (7).

Causal pair 2. client-1 second SEND (L=6) → client-2 RECV (L=16). Chain: client-

1 SEND (6) → replica-A RECV (10) → APPLY (11) → SEND (12) → RECV (13) → APPLY (14) → SEND (15) → client-2 RECV (16).

Concurrent pair. client-2 SEND (L=1) and client-1 SEND (L=6): no message chain connects them in either direction. The values differ ($1 < 6$), but that does not mean client-2 acted first: Lamport clocks only guarantee that if a happened before b, then $L(a) < L(b)$.

Limitation. Lamport values cannot tell causally related events from concurrent ones. Vector clocks can: a happened before b exactly when $V(a) < V(b)$, and incomparable vectors mean concurrent events.

Appendix B

Reflection Questions

1. In both cases the client receives nothing, and it cannot see the server's state. A retry with the same key is harmless: if the first request was applied, the replica answers from its deduplication store; if not, it applies it now. The counter changes once either way (TC-05, TC-14).

2. client-2 SEND (L=1) and client-1 SEND (L=6) are concurrent. client-1's value is larger only because it had already exchanged messages with the replica. No message links the two events, so either could have happened first in real time.

3. One failed replica of any kind (crash before or after apply, slow reply, restart) is tolerated: each committed write is on at least two replicas, and any two majorities overlap (TC-06, TC-12, TC-14 to TC-17). With two replicas down, writes stop committing: liveness is lost, but safety holds because the client reports failure (TC-07). The guarantee breaks if two replicas lose their memory, for example both restart, which is a safety violation.

4. Append each applied operation (key, counter, value) to a file on disk and flush it before replying; on start, replay the file. This anticipates Raft's persistent log (Week 6).

Appendix C

Captured Event Log (logs/b2_merged.log)

[client-1] SEND	Increment(counter=x, delta=1)	L=1
[client-2] SEND	Increment(counter=y, delta=1)	L=1
[client-3] SEND	Get(counter=x)	L=1
[replica-A] RECV	Increment(counter=x, delta=1)	L=2 (received L=1)
[replica-A] APPLY	counter=x -> 1	L=3
[replica-A] SEND	IncrementReply(new_value=1)	L=4
[client-1] RECV	IncrementReply(new_value=1)	L=5 (received L=4)
[replica-A] RECV	Get(counter=x)	L=5 (received L=1)
[client-1] SEND	Increment(counter=x, delta=1)	L=6
[replica-A] SEND	GetReply(value=1, found=True)	L=6
[client-3] RECV	GetReply(value=1, found=True)	L=7 (received L=6)
[replica-A] RECV	Increment(counter=y, delta=1)	L=7 (received L=1)
[client-3] SEND	Get(counter=x)	L=8
[replica-A] APPLY	counter=y -> 1	L=8
[replica-A] SEND	IncrementReply(new_value=1)	L=9
[client-2] RECV	IncrementReply(new_value=1)	L=10 (received L=9)
[replica-A] RECV	Increment(counter=x, delta=1)	L=10 (received L=6)
[client-2] SEND	Increment(counter=y, delta=1)	L=11
[replica-A] APPLY	counter=x -> 2	L=11
[replica-A] SEND	IncrementReply(new_value=2)	L=12
[client-1] RECV	IncrementReply(new_value=2)	L=13 (received L=12)
[replica-A] RECV	Increment(counter=y, delta=1)	L=13 (received L=11)
[replica-A] APPLY	counter=y -> 2	L=14
[replica-A] SEND	IncrementReply(new_value=2)	L=15
[client-2] RECV	IncrementReply(new_value=2)	L=16 (received L=15)
[replica-A] RECV	Get(counter=x)	L=16 (received L=8)
[replica-A] SEND	GetReply(value=2, found=True)	L=17
[client-3] RECV	GetReply(value=2, found=True)	L=18 (received L=17)

Appendix D

Package Versions (pip freeze)

```
grpcio==1.84.0  
grpcio-tools==1.84.0  
iniconfig==2.3.0  
packaging==26.3  
pluggy==1.6.0  
protobuf==7.36.2  
Pygments==2.21.0  
pytest==9.1.1  
setuptools==84.0.0  
typing_extensions==4.16.0
```